SIH 2026 — PS 26070: Team Structure & Work Split

AI/ML system for identification, classification, and prediction of tropical cyclone patterns using multi-source satellite data — Ministry of Earth Sciences (MoES) / IMD

Final team structure — choose what works, this is the proposed plan.

Problem Statement Alignment Notes

Added after review against PS 26070 to close three gaps between our build plan and the exact wording of the problem statement. These apply across the roles below and should be reflected in the final report/pitch, not just the code.

■ 1. "Cyclone patterns" — broadened beyond track/intensity

- The PS says **patterns**, not just track + intensity. In addition to detection, classification, and forecasting, briefly cover: cyclone **genesis/formation signals** (favorable SST + low wind shear conditions before a system forms), and basic **structural pattern** tagging from IR imagery (e.g. eye present / no eye, spiral-banding, curved-band vs. shear pattern) using established satellite classification conventions such as the **Dvorak technique**.
- This does not need a full new model — Person 3 can add structural pattern as an auxiliary output/tag alongside category and wind speed, and Person 1 can flag pre-genesis environmental windows in the dataset. Even a lightweight version is enough to demonstrate "patterns" was addressed, not just track.

■ 2. Near-real-time framing (not just historical/retrospective)

- IBTrACS is a historical, post-season dataset — fine for training, but the demo must not look like it only replays old storms. Person 1's fallback (preprocessed INSAT set) is sensible for the sprint, but the pipeline and inference functions should be written so a **live/near-real-time INSAT frame** could be dropped in later without a redesign.
- Frontend (Person 5) should explicitly label the dashboard as supporting near-real-time monitoring, e.g. a header note: "Live/near-real-time cyclone monitoring — currently running on latest available satellite pass." This directly matches how IMD actually operates (near-real-time nowcasting), which is the department listed on the PS.

■ 3. Align classification scale with IMD's official scheme

- Person 3's classes must exactly match the **IMD / RSMC New Delhi** cyclone intensity scale: Depression → Deep Depression → Cyclonic Storm → Severe Cyclonic Storm → Very Severe Cyclonic Storm → Extremely Severe Cyclonic Storm → Super Cyclonic Storm, with the standard wind-speed bands IMD uses for each category.
- Do not substitute the Saffir-Simpson (US) scale or invent custom bins — since the organization on the PS is IMD, judges will check this. State explicitly in the final report/README that the classification scheme is IMD-aligned, with a source citation to RSMC New Delhi criteria.

Team Summary

Role	Owns	Output
Person 1 — Data Engineer	Collect & preprocess IBTrACS + INSAT-3D/3DR + ERA5 data	ML-ready datasets + preprocessing scripts
Person 2 — Cyclone Detection	Detect cyclone presence & location in a satellite image	Detection model + confidence + coordinates/bbox
Person 3 — Classification & MTCSWA	Map to agency category/intensity + wind speed (+ structural category tag)	Category + wind speed + pressure + confidence
Person 4 — Forecasting	Predict future track + intensity for 6/12/24h from past movement + environment	Future lat/lon + predicted wind speed
Person 5 — Frontend/UI	React + Leaflet dashboard: imagery, current cyclone, classification, predicted track, uncertainty, landfall, risk	Complete frontend integrated with backend + ML models

WORKS IN DETAIL

Person 1 — Data Engineer

Your responsibility

You are responsible for building the complete data foundation for the project. Your output will be used by Person 2 (Detection), Person 3 (Classification), Person 4 (Forecasting), and final integration.

Your goal: raw multi-source data → clean, synchronized, ML-ready dataset.

1. Dataset sources

Do NOT spend several days randomly searching for datasets. Start with these sources.

A. NOAA IBTrACS — Primary cyclone track dataset

Use IBTrACS for historical cyclone information. We need:

• Cyclone/storm ID

• Cyclone name if available

• Date/time

• Latitude

• Longitude

• Maximum sustained wind

• Minimum pressure

• Basin

• Storm classification/category where available

This will be our historical ground truth for cyclone tracks and intensity.

B. INSAT-3D / INSAT-3DR — Satellite imagery

Try to obtain: INSAT-3D imagery, INSAT-3DR imagery, Infrared imagery, Visible imagery if available. Priority is IR imagery, because cyclone structure can be observed effectively from infrared satellite observations. Preferred source: MOSDAC / official Indian satellite data sources. If accessing raw data takes too long, use a suitable preprocessed INSAT-3D/3DR cyclone dataset as a fallback. Do not spend 3–4 days fighting with a difficult data portal.

> **■ Design for near-real-time, even if the sprint build uses historical data**
>
> - Write the ingestion/preprocessing functions so a single incoming INSAT frame can be pushed through the same pipeline used for historical batches (same normalization, cropping, resizing steps) — don't hardcode batch-only assumptions.
> - Log the observation timestamp alongside every record so the frontend can display "latest available pass" instead of implying a fixed historical replay.

C. ERA5 — Environmental data

Use ERA5 for environmental variables. Initially prioritize: Sea Surface Temperature, Sea-level pressure, U-component wind, V-component wind. If time permits: Temperature, Humidity, Geopotential variables. Don't try to download every ERA5 variable.

2. Create a master dataset

The most important thing you have to build is a unified table, e.g. cyclone_id, timestamp, latitude, longitude, wind_speed, pressure, category, satellite_image, sst, u_wind, v_wind. The exact variables depend on what is actually available.

Example:

cyclone_id	timestamp	latitude	longitude	wind_speed	pressure	category
TC001	2020-11-25 06:00	15.42	82.31	130	960	Severe Cyclonic Storm

3. Synchronize the datasets

This is VERY important. IBTrACS, INSAT and ERA5 won't necessarily have exactly the same timestamps, spatial resolution, coordinate systems, or file formats. Create a pipeline: IBTrACS observation → find matching satellite observation → find ERA5 environmental values → create one training sample. Your final sample should represent what the cyclone looked like + where it was + how strong it was + what the environment was like at that time.

4. Satellite preprocessing

Create scripts to read satellite images, handle missing files, convert formats if necessary, crop relevant regions, resize images, normalize images, remove corrupted samples, and associate images with cyclone IDs and timestamps.

```
src/data/preprocess_satellite.py
```

5. Create three ML datasets

Dataset A — Detection: satellite image → cyclone location/bounding box (Person 2).

Dataset B — Classification: satellite image + environmental variables → cyclone category/intensity (Person 3).

Dataset C — Forecasting: temporal sequences t-24…t → t+6/t+12/t+24, each observation containing latitude, longitude, wind_speed, pressure, sst, u_wind, v_wind (Person 4).

6. Train/validation/test split

Do NOT randomly split individual images if that causes the same cyclone to appear in both training and test data. Prefer splitting by cyclone/event, e.g. Train: Cyclones A–H, Validation: I–J, Test: K–L. This gives a more meaningful evaluation.

7. Data visualization

• Number of cyclones

• Number of satellite images

• Cyclone categories
• Wind-speed distribution
• Pressure distribution
• Geographic distribution
• Sample satellite images
• Example cyclone tracks
• Missing data statistics

8. Required final folder

```
data/
■■■ raw/ (ibtracs, insat, era5)
■■■ processed/ (detection, classification, forecasting)
■■■ metadata/ (master_dataset.csv, train.csv, validation.csv, test.csv)
```

9. Day-by-day target

Day	Target
1	Get IBTrACS; find INSAT data; set up ERA5 access; understand formats
2	Download/sample data; build metadata; start satellite preprocessing
3	Align IBTrACS + satellite data; start ERA5 alignment
4	Complete master dataset
5	Detection dataset; classification dataset; forecasting sequences
6	Train/validation/test split; PyTorch Dataset/DataLoader
7	Validate everything; fix missing/incorrect samples
8	FINAL HANDOFF: dataset, metadata, preprocessing scripts, DataLoader, train/test split, sample data, documentation

Person 2 — Cyclone Detection

Your responsibility

Your job: given a satellite image, identify whether a tropical cyclone exists and determine its location. You are responsible only for identification/detection.

1. Input

Primary input: INSAT satellite image. Potentially: IR image, Visible image, Multispectral image — depending on what Person 1 provides.

2. Output

Your model must identify cyclone present/not present, location, bounding box if possible, and confidence, e.g.:

```
{ "detected": true, "confidence": 0.94, "center": {"latitude": 16.52, "longitude": 82.31},
"bbox": [420,190,600,370] }
```

■ Also tag basic structural pattern (covers "patterns" from the PS)

- When a cyclone is detected, optionally emit a lightweight **structural_pattern** field alongside the bounding box — e.g. "eye_visible", "curved_band", "shear_pattern", "no_organized_center" — based on simple IR-image cues (this can be a small auxiliary classifier head or even a rule-based check on the cropped region, it does not need to be sophisticated).
- This gives the system an explicit answer to "cyclone patterns," not just a bounding box, without adding real project risk.

3. Model strategy

Do not build an object detector from scratch. Start with a pretrained detection model (e.g. YOLO or another suitable pretrained detection architecture), then fine-tune it on the cyclone dataset.

4. Dataset preparation

Work with Person 1 to ensure labels contain image, cyclone location, bounding box. If the dataset doesn't have bounding boxes, investigate whether the cyclone center can be converted into an appropriate detection target.

5. Training

Start with a baseline, then experiment with image resolution, augmentation, learning rate, batch size, number of epochs, pretrained weights. Don't spend the entire sprint hyperparameter tuning.

6. Evaluation

You MUST calculate metrics — at minimum Precision, Recall, IoU, mAP — and show sample predictions visually.

7. Geographic conversion

If your detector outputs pixel coordinates, convert: pixel coordinates → image geographic bounds → latitude/longitude. Create a reusable function for this.

8. Inference function

result = detect_cyclone(image) — should return the standardized result. Don't make me understand your entire training code just to run inference.

9. Required handoff

```
models/detection/model_weights.pt
src/detection/detector.py, inference.py
Also: requirements.txt, sample_input, sample_output, metrics, README
```

10. Day-by-day

Day	Target
1	Prepare dataset; set up YOLO/PyTorch; test pretrained model
2	Convert labels; train baseline
3	Improve training; generate predictions
4	Evaluate model
5	Improve detection
6	Geographic coordinate conversion; build inference function
7	Final model; final metrics; test inference
8	FINAL HANDOFF — no major new features after this

Person 3 — Classification + Intensity

Your responsibility

Your job: determine what type/intensity of cyclone has been detected and estimate its strength.

1. Input

Use satellite image + SST + U/V wind + pressure where available. Ideal architecture: image features + environmental data → feature fusion → classification (category) and regression (wind speed) heads.

2. Classification

Do NOT create artificial classes just to make the project look complex — use the actual classification scheme supported by your training labels.

> **■ Use IMD's official classification scale (RSMC New Delhi), not a custom or Saffir–Simpson scale**
>
> - Since the department on the PS is IMD, the category labels must be the standard IMD/RSMC New Delhi scale: **Depression** → **Deep Depression** → **Cyclonic Storm** → **Severe Cyclonic Storm** → **Very Severe Cyclonic Storm** → **Extremely Severe Cyclonic Storm** → **Super Cyclonic Storm**, each tied to IMD's standard sustained wind-speed bands.
> - If IBTrACS labels use a different convention for some basins, remap them to the IMD scale before training (or restrict training data to North Indian Ocean storms, where IBTrACS already reports IMD-style categories).
> - State this alignment explicitly in the README/report with a citation to RSMC New Delhi criteria — this is an easy, high-value point for judges from MoES/IMD to notice.

3. Intensity prediction

Predict maximum sustained wind speed (MUST). Optionally predict minimum pressure (NICE TO HAVE) — if pressure prediction isn't reliable, don't sacrifice the main model for it.

4. Output

```
{ "category": "Severe Cyclonic Storm", "wind_speed": 145, "pressure": 950, "confidence": 0.91
}
```

5. Metrics

Classification: Accuracy, Precision, Recall, F1-score, Confusion matrix. Wind: MAE, RMSE. Pressure (if implemented): MAE, RMSE.

6. Compare image-only vs multi-source if possible

Model A: satellite image → prediction. Model B: satellite image + ERA5/SST → prediction. Show whether environmental information improves performance — this directly supports the multi-source aspect of the problem statement.

7. Inference function

result = classify_cyclone(image, environmental_data) — return the standard JSON structure.

8. Required handoff

```
models/classification/classifier.pt, intensity_model.pt
src/classification/classifier.py, inference.py
Also: README, requirements, metrics, sample input, sample output
```

9. Day-by-day

Day	Target
1	Define classes (IMD scale); prepare labels; set up model
2	Train baseline
3	Evaluate classification
4	Build wind regression
5	Add environmental features
6	Compare image-only vs multi-source
7	Finalize models; metrics; inference
8	FINAL HANDOFF

Person 4 — Cyclone Forecasting

Your responsibility

You own the most important prediction component: predict the future trajectory and intensity of the cyclone.

1. Input

Use a historical sequence (t-24h, t-18h, t-12h, t-6h, t). Each timestep can contain latitude, longitude, wind_speed, pressure, sst, u_wind, v_wind.

2. Output

Predict +6h, +12h, +24h — at each point latitude, longitude, wind_speed, e.g.:

```
{ "forecast": [ {"hours": 6, "latitude": 17.1, "longitude": 81.6, "wind_speed": 150},
{"hours": 12, "latitude": 17.8, "longitude": 80.9, "wind_speed": 155}, {"hours": 24,
"latitude": 19.1, "longitude": 79.5, "wind_speed": 160} ] }
```

3. First build a baseline

Before your neural network, implement a persistence/movement-vector model using recent movement to estimate future position. This gives Baseline vs AI prediction — important for demonstrating the AI actually improves over a simple approach.

4. AI model

Start with LSTM or another lightweight temporal model. Do NOT start with a huge Transformer. Architecture: past observations → LSTM → future coordinates + future wind.

5. Multi-source forecasting

After the basic LSTM works, add historical track + wind + pressure + SST + U/V wind — this is where the multi-source requirement becomes particularly useful.

6. Evaluation

Track prediction error: geographic distance between predicted and actual position, reported as 6h/12h/24h Track Error in kilometres. Intensity: Wind MAE, Wind RMSE.

7. Inference function

result = forecast_cyclone(history) — return 6h, 12h, 24h.

8. Required handoff

```
models/forecasting/forecast_model.pt
src/forecasting/forecaster.py, inference.py
Also: input format, output format, metrics, sample input, sample output, README, requirements
```

9. Day-by-day

Day	Target
1	Build sequence dataset; understand features
2	Persistence baseline
3	LSTM baseline
4	Train model
5	6-hour prediction
6	12/24-hour prediction
7	Environmental feature fusion; evaluation; finalize model

Day	Target
8	FINAL HANDOFF

Person 5 — UI/Frontend

Your responsibility

You are responsible only for the frontend/UI. You don't need to train models. You don't need to build FastAPI. You don't need to understand the internal model architecture.

Your job: create a professional dashboard that can display the output of the AI system.

1. Technology

React, Vite, JavaScript/TypeScript, Leaflet, CSS/Tailwind if preferred. Keep it simple and fast.

2. Main dashboard

Sections: interactive map (historical / current / forecast track), detection card, classification card, forecast card, landfall panel, AI risk indicator.

> **■ Header should state near-real-time framing explicitly**
>
> - Add a small status line under the dashboard title, e.g. "Near-real-time monitoring — showing latest available satellite pass" (with a mock timestamp for now), so the demo visibly matches how IMD actually operates rather than reading as a historical replay tool.

3. Interactive map

Use Leaflet. Map should eventually support historical track, current position, forecast, and an uncertainty polygon/buffer around the forecast track. Initially use mock coordinates.

4. Satellite viewer

Let the user upload/select a satellite image, view it, zoom, and see cyclone detection. Prepare the UI to support bounding-box visualization.

5. Detection card

Show: Cyclone Detected ✓, Confidence 94%, Location 16.52°N 82.31°E.

6. Classification card

Show: Category (Severe Cyclonic Storm — IMD scale), Wind 145 km/h, Pressure 950 hPa, Confidence 91%. Optionally show the structural pattern tag from Person 2 (e.g. "eye visible") as a small badge.

7. Forecast card

Show 6 / 12 / 24 hour predicted positions.

8. Landfall panel

Estimated landfall location, estimated time, predicted wind. Initially use mock data.

9. AI Risk Indicator

Display a 0–100 risk score with a HIGH/MEDIUM/LOW band. Make it clear in the UI: "AI-derived risk indicator — not an official warning."

10. Charts

Wind speed over time, pressure over time, forecast track, model confidence. Don't overdo charts — the map is the main visualization.

11. Mock API

Before the backend is ready, use dummy JSON so the UI can be finished independently.

12. API structure

Prepare the frontend so mockData can later be swapped for /api/analyze. Don't hardcode model results throughout the UI.

13. Day-by-day

Day	Target
1	React setup; UI structure
2	Dashboard
3	Leaflet map; historical/current/forecast track
4	Satellite image viewer
5	Detection/classification cards
6	Forecast + landfall + risk panels
7	Charts; responsive UI; mock API
8	FINAL HANDOFF

Team Integration Contract

Person 1 → Data

Common ML-ready dataset fields: image, timestamp, cyclone_id, latitude, longitude, wind_speed, pressure, sst, wind_u, wind_v, category. Primarily for the ML team — the frontend does not directly consume Person 1's dataset.

Person 2 → Detection

```
{ "detected": true, "latitude": 16.52, "longitude": 82.31, "confidence": 0.94, "bbox":
[420,190,600,370], "structural_pattern": "eye_visible" }
```

Person 3 → Classification

```
{ "category": "Severe Cyclonic Storm", "wind_speed": 145, "pressure": 950, "confidence": 0.91
} // category values follow the IMD/RSMC New Delhi scale
```

Person 4 → Forecast

```
{ "forecast": [ {"hours": 6, "latitude": 17.1, "longitude": 81.6, "wind_speed": 150},
{"hours": 12, "latitude": 17.8, "longitude": 80.9, "wind_speed": 155}, {"hours": 24,
"latitude": 19.1, "longitude": 79.5, "wind_speed": 160} ] }
```

Person 5 — Frontend

For development, the frontend should be prepared to consume /api/detect, /api/classify, /api/forecast. The final application should primarily consume /api/analyze, because all model outputs are combined in the backend.

The final response looks like:

```
{ "detection": {...}, "classification": {...}, "forecast": [...], "landfall": {"estimated":
true, "latitude": 18.42, "longitude": 84.91, "estimated_time": "..."}, "risk": {"score": 82,
"level": "HIGH"} }
```

The frontend maps these fields to: detection → detection card + current cyclone marker; classification → intensity card; forecast → forecast track on map; landfall → landfall panel + map; risk → risk indicator.